

Интеграция Great Expectations в Airflow

Цель видео

Научиться интегрировать GE для проверки качества данных в Airflow.

Проблемы ETL, построенных на Airflow

Зачем интегрировать Great Expectations с Airflow?

- 1 Автоматизация проверок качества данных в ETL-процессах
- 2 Обеспечение качества данных на каждом этапе обработки
- 3 Облегчение мониторинга и оповещений о проблемах с качеством данных

Шаги интеграции

- 1 Настройка окружения
- 2 Конфигурирование Great Expectations
- 3 Подготовка ожиданий
- 4 Интеграция с Airflow
- 5 Создание чекпоинтов
- 6 Запуск

Настройка окружения

- 1 Установленный Python
- 2 Установленный и настроенный Airflow
- 3 Установленная библиотека Great Expectations

```
pip install great_expectations
```

```
Requirement already satisfied: asttokens in c:\users\vadim\anaconda3\lib\site-packages (from stack-data->Ipython>=7.16.3->great_expectations) (2.0.5)
Requirement already satisfied: pure-eval in c:\users\vadim\anaconda3\lib\site-packages (from stack-data->Ipython>=7.16.3->great_expectations) (0.2.2)
Requirement already satisfied: executing in c:\users\vadim\anaconda3\lib\site-packages (from stack-data->Ipython>=7.16.3->great_expectations) (0.8.3)
Requirement already satisfied: anyio<4,>=3.1.0 in c:\users\vadim\anaconda3\lib\site-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook>=6.4.10->great_expectations) (3.5.0)
Requirement already satisfied: websocket-client in c:\users\vadim\anaconda3\lib\site-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook>=6.4.10->great_expectations) (0.58.0)
Requirement already satisfied: soupsieve>1.2 in c:\users\vadim\anaconda3\lib\site-packages (from beautifulsoup4->nbconvert>=5->notebook>=6.4.10->great_expectations) (2.3.2.post1)
Requirement already satisfied: webencodings in c:\users\vadim\anaconda3\lib\site-packages (from bleach->nbconvert>=5->notebook>=6.4.10->great_expectations) (0.5.1)
Requirement already satisfied: sniffio>=1.1 in c:\users\vadim\anaconda3\lib\site-packages (from anyio<4,>=3.1.0->jupyter-server>=1.8->nbclassic>=0.4.7->notebook>=6.4.10->great_expectations) (1.2.0)
Installing collected packages: makefun, tzlocal, altair, great_expectations
Successfully installed altair-4.2.2 great_expectations-0.18.19 makefun-1.15.4 tzlocal-5.2
(base) PS C:\Users\Vadim\PycharmProjects\skillox\tutorial-great-expectations\great_expectations>
```


Конфигурирование Great Expectations

```
great_expectations init
```

```
-----  
/ _ _ _ _ _ | | | _ _ _ _ _ | | | _ _ _ _ _  
| ( | ' / - ) ' | | | \ \ / ' V - ) | _ / ' | | / _ \ ' \ C - <  
 \ _ _ | \ _ _ \ , \ _ _ | | _ _ / \ \ . _ _ \ _ _ \ \ _ _ \ , \ _ _ | \ _ _ / | | / _ _/  
          | |  
          ~ Always know what to expect from your data ~
```

Let's create a new Data Context to hold your project configuration.

Great Expectations will create a new directory with the following structure:

```
great_expectations  
|-- great_expectations.yml  
|-- expectations  
|-- checkpoints  
|-- plugins  
|-- .gitignore  
|-- uncommitted  
    |-- config_variables.yml  
    |-- data_docs  
    |-- validations
```

Подготовка ожиданий

```
import great_expectations as ge
```

```
df = ge.from_pandas(your_dataframe)
```

```
df.expect_column_values_to_not_be_null('column_name')  
df.save_expectation_suite('your_expectation_suite')
```

Интеграция с Airflow

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
import great_expectations as ge

def validate_data(**kwargs):
    context = ge.data_context.DataContext('path/to/great_expectations')
    result = context.run_checkpoint(checkpoint_name='your_checkpoint')
    return result['success']

default_args = {
    'owner': 'airflow',
    'start_date': '2024-07-30',
}

dag = DAG('data_validation', default_args=default_args, schedule_interval='@daily')

validate_task = PythonOperator(
    task_id='validate_data',
    python_callable=validate_data,
    dag=dag
)
```


Создание чекпоинтов

```
context.add_checkpoint(  
    name="your_checkpoint",  
    config_version=1,  
    class_name="Checkpoint",  
    validations=[  
        {  
            "batch_request": {  
                "datasource_name": "your_datasource",  
                "data_connector_name": "your_data_connector",  
                "data_asset_name": "your_data_asset"  
            },  
            "expectation_suite_name": "your_expectation_suite"  
        }  
    ]  
)
```

Итог видео

Научились интегрировать GE для проверки качества данных в Airflow.